

Artificial Intelligence Project

Marco Ballarin

September 2026

1 Introduction

This report describes the pipeline that was built to predict the outcome of a tennis match. We will describe the critical choices that were made in every step of the pipeline, from the loading of the data to the evaluation of the model.

The notebooks generated describing in detail the steps, are available in the `notebooks` folder, in particular, the notebook `notebooks/preprocess.ipynb` contains the description of the step of the pipeline until the feature engineering, while the other notebooks contain the training and evaluation of three models: random forest, xgboost and logistic regression.

2 Data Loading

The source of the dataset provides one spreadsheets per season containing the results of every match played in that season. The function `load_data`, defined in `src/data/loader.py`, is responsible for turning this collection of yearly files into a single `DataFrame`. The matches are sorted in chronological order into the resulting `DataFrame`, because tennis matches are time dependent, thus, the outcome of a match can only be influenced by matches that have already been played. Consequently, every information that we want to build (recent form, head-to-head records, surface-specific results) must use only matches that have already been played. Working on a chronologically ordered dataset is therefore a requirement because every later stage of the pipeline relies on it.

3 Label Creation

In the raw dataset every match is described from the point of view of its outcome: there is a `Winner` and a `Loser` column, and each statistic appears twice, once for the winner and once for the loser (e.g. `WRank/LRank`, `B365W/B365L`, `Wsets/Lsets`). Such a representation cannot be used for learning, because the target is implicitly encoded in the position of the columns, hence, a model will always predict that the player appearing in the winner column is the winner, without learning anything about the game.

The function `create_labels`, defined in `src/data/preprocess.py`, solve the problem by rewriting each match in terms of two neutral players (`player_1` and `player_2`). For every row a fair coin is tossed, using a random generator, and the winner is placed in `player_1` or in `player_2` accordingly, the loser taking the remaining slot. The same swap is applied to every pair of winner/loser statistics, which are renamed into the corresponding `player1_/player2_` columns, so the consistency of each row is maintained. The target column `winner` is then set to 1 when `player1` is the actual winner of the match and to 0 otherwise.

Finally, the original `Winner`, `Loser` and paired statistic columns are dropped, with the only exception of the name of the actual winner, which is kept in the `match_winner` column, because it will be used later for feature engineering.

4 Data Splitting

The function `train_test_split`, defined in `src/data/loader.py`, divides the dataset into a training and a test set by choosing a proportion of rows to keep for testing. The split isn't performed randomly, in contrast, it follows the chronological order of the matches. This avoid data leakage, because it guarantees that the training set contains only matches that occurred before those in the test set.

5 Preprocessing

The function `preprocessing_pipeline`, defined in `src/data/preprocess.py`, builds the `Pipeline` that turns the raw matches into a dataset ready for feature engineering, without any missing values. The pipeline contains several transformers, that perform the following operations:

- **Rows Filter:** Matches are filtered to remove rows that do not contain useful information for the model, for instance matches not played at all, such as walkovers, disqualifications, or matches that do not provide any set information. These matches can be considered as outliers, because they represnt a very small fraction of the dataset and they do not provide any useful information for the model.
- **Fix Typos:** The `Comment` column contains typos in the data. The `ReplaceValuesTransformer` replace the value of "Rretired" with the correct form "Retired".
- **Features Drop:** Most of the odds columns have a large number of missing values, therefore they are removed, because imputing them would mean inventing almost the whole column.
- **Missing Value Imputation:** The other columns that contain missing values are imputed with a strategy specific to each column:
 - **Games and sets** are filled with 0, because a missing game score means that the set was simply never played.
 - **Odds** are filled with 2.0 equal for both players. This strategy can be applied because the rows of the dataset with no values on the odds columns, they contains missing values for the odds of both players.
 - **Rank & Points** are filled in the `RankPointsImputer`, that insert for ranks and points values with the last observed value for the player, if any, otherwise with the median rank or median points.
 - **Best Of** is computed by the `BestOfImputer` using the number of sets played
- **Transformations:** Finally, the categorical columns are encoded as follows:

- **Tournament, Court, Surface** are one-hot encoded. However, the original columns are kept, because they will be used later for building features that are specific to the tournament, court or surface of the match.
- **Round** is ordinally encoded following the real progression of a tournament (Round Robin, 1st Round, 2nd Round, 3rd Round, 4th Round, Quarterfinals, Semifinals, The Final).
- **Series** is ordinally encoded following the importance of the tournament (ATP250, ATP500, Masters 1000, Masters Cup, Grand Slam).

6 Feature Engineering

The creation of features is implemented in the `feature_engineering_pipeline`, defined in `src/features/features.py`, which builds the `Pipeline` that turns the preprocessed matches into the final set of features. The pipeline contains a single step which applies the `FeatureEngineeringTransformer`.

The columns produced by the preprocessing describe a match as it ended with information such as the games won in each set, the number of sets, the closing odds. Taken alone they say almost nothing about the two players facing each other. Therefore, the purpose of this process is to replace this description with a set of features that summarise the history of the two players up to the moment the match starts, and keeping some information about the match itself, such as the surface and the court. To do so the transformer keeps a state made of two objects:

- `players_`, a dictionary that maps the name of a player to a `Player` object (`src/utils/player.py`), which stores everything that is known about that player so far.
- `matches_`, the list of `Match` records (`src/utils/match.py`) already built, one for every processed row.

The construction of the features is delegated to the method `create_features`, which iterates over the rows of the `DataFrame` in chronological order and for each row it computes the features of the match using the current state of the two players, and then it updates the state of the players with the outcome of the match. The update is performed differently depending on whether the player won or lost the match, and this is the reason why the `match_winner` column was kept during label creation.

The complete list of features is reported in to the relative section on the notebook `notebooks/preprocess.ipynb`.

7 Fine Tuning

Pipeline hyperparameter tuning is implemented in the function `fine_tune`, defined in `src/training/training.py`, which supports the two search strategies

offered by scikit-learn `GridSearchCV` and `RandomizedSearchCV`. Because match outcomes are inherently time-dependent, the training-validation split must preserve chronological order rather than splitting randomly. Because match outcomes are inherently time-dependent, the training-validation split must preserve chronological order rather than splitting randomly. Then an analysis of the feature importance of the fitted pipeline is performed, and finally recursive feature elimination is applied to the pipeline using `RFECV` to identify the optimal subset of features.

8 Evaluation

During the evaluation phase, the optimal estimator found so far is selected based on the cross-validation scores and model simplicity. The full pipeline is used to predict the outcome of the matches in the test set. Then we evaluate the generalization ability of the model by computing different metrics. Since the task is a binary classification problem between two players, the accuracy and roc-auc score are the most important metrics to consider. Because respectively the accuracy measures the proportion of correct predictions, while the roc-auc score measures the ability of the model to distinguish between the two classes. The results are reported in the **Evaluation** sections of the notebooks dedicated to the training of each model.

In conclusion an error analysis examines the distribution of misclassified matches in order to identify systematic patterns or factors that may contribute to prediction difficulty. The analysis shows that the models tend to struggle particularly with matches in which the most influential features favor the player who ultimately loses. For example, in the Random Forest model, features such as ranking and ranking points can strongly favor one player, even when the match outcome is ultimately different from the prediction. Therefore, the findings indicate that the models have limitations in capturing the full complexity of tennis matches and the various factors that can influence their outcomes.

9 Results

The evaluation results for the three models are reported in detail in the corresponding notebooks. Overall, the three approaches achieved very similar performance, suggesting that none of the models provides a substantial advantage over the others in terms of predictive accuracy. Among the evaluated models, Logistic Regression, Random Forest and XGBoost achieved the same generalization accuracy, with an accuracy of 0.68. This is also reflected in the ROC-AUC scores, which were almost identical across all three models, with each model achieving a score of approximately 0.74.

Predicting the outcome of tennis matches is a particularly challenging task, as match results depend on a wide range of factors that are not necessarily captured by the available dataset. In addition to measurable player and match

statistics, factors such as players' mental and physical condition, weather conditions, crowd support, playing conditions, and other contextual aspects can have a significant impact on the final outcome. Consequently, some degree of uncertainty is unavoidable in this prediction task.

Considering these limitations, the obtained results can be considered satisfactory, with the models correctly predicting approximately two out of every three matches in the generalization set. However, the relatively small differences between the three models suggest that further improvements are likely to depend more on the quality and richness of the input features than on the choice of classification algorithm alone.